

Evaluators

CMSC 323: Design and implementation of programming languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in Programming Languages: Application and Interpretation.

Evaluators

A programming language is an evaluator. It reduces a program to a value (something that does not need to be further evaluated).

Implementation Strategies

There are two major implementation strategies: compilers and interpreters.

Interpreters

- ◆ Receives a program and produces a value

Compilers

- ◆ Consumes a program and produces another program

Implementation Strategies

- ◆ *Most languages do not dictate implementation.*
- ◆ There are no hard distinctions between interpreters and compilers for many programming languages.

Interactive vs Non-interactive PLs

- ◆ An implementation might be interactive or non-interactive
- ◆ An interactive program is not necessarily an interpreted program
- ◆ It is easier to implement interactive interpreters
- ◆ REPL — Read-Eval-Print-Loop

Just In Time Compilation

- ◆ **JIT** – Many languages now have JIT Compilation – which is just in time compilation. The evaluator starts out as an interpreter and if it finds that it is interpreting the same code repeatedly, it uses the compiled code.

Interpreters

Why Interpreters?

- ◆ A simple interpreter is easier to write than a compiler.
- ◆ Debugging an interpreter is easier than debugging a compiler.
- ◆ An interpreter makes use of the underlying language instead of building features from scratch.
 - In Practice: A compiler often has to build features (e.g., Objects) from scratch at a low level (e.g., assembly). An interpreter often uses the language in which the interpreter is implemented to handle the heavy lifting. E.g., an interpreter for Python written in Java could map Python objects to Java objects and execute those, never having to generate assembly code.

Designing and Implementing an Evaluator

Let's write a simple evaluator that does simple arithmetic. Our language will have:

- ◆ Numbers
- ◆ Some arithmetic (addition for now)

Standard Implementation Plan

We use an **abstract syntax** to represent our program

- ◆ An abstract syntax is an in-computer representation of programs
- ◆ An abstract syntax allows us to represent the essence of the input, ignoring superficial syntactic details

Example

{+ 1 2}

Program

AST

Result

Parser

Interpreter

Language used

Object
Language

Meta Language

Meta Language

Example Languages

DrRacket

Scheme

Scheme

Java

C, C++ and other languages

C, C++ and other languages

Python

C

C

Assembly

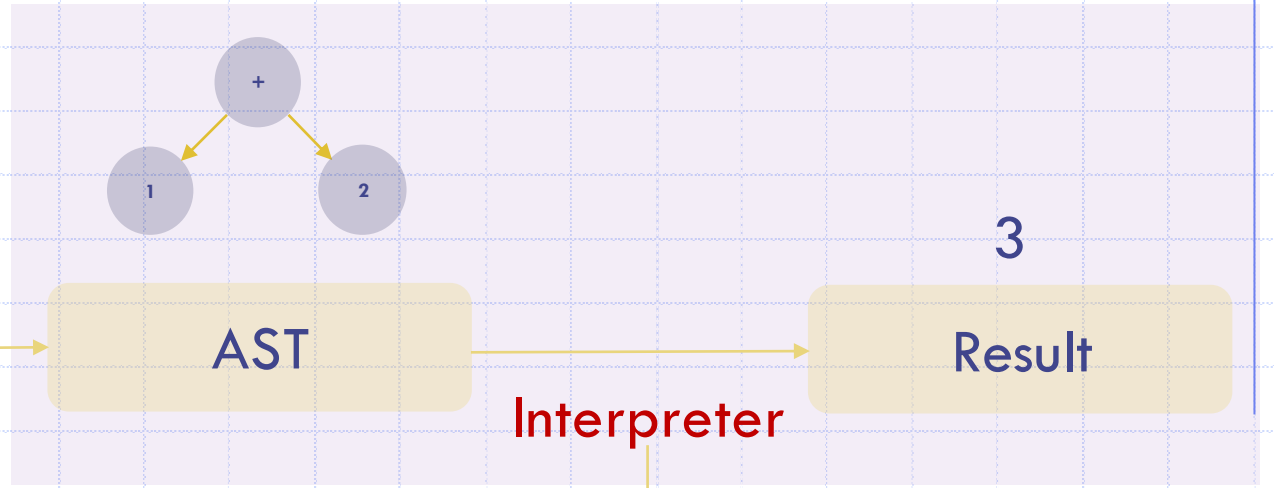
Machine Code

Machine Code

C

Assembly Language

Assembly Language



Representing Abstract Syntax

- ◆ Numbers are represented as leaves in our tree.
- ◆ Operations are represented as nodes in our tree.

```
(define-type AE  
  [num (n : Number)]  
  [plus (lhs : AE) (rhs : AE)]  
  [sub (lhs : AE) (rhs : AE)])
```

Representing Abstract Syntax

```
(define-type AE  
  [num (n : Number)]  
  [plus (lhs : AE) (rhs : AE)]  
  [sub (lhs : AE) (rhs : AE)])
```

- We define a new type AE (an arithmetic expression)
- An arithmetic expression can either be a num, a plus or a sub

Representing Abstract Syntax

```
(define-type AE  
  [num (n : Number)]  
  [plus (lhs : AE) (rhs : AE)]  
  [sub (lhs : AE) (rhs : AE)])
```

- A num takes one argument that must be an actual number

Representing Abstract Syntax

```
(define-type AE  
  [num (n : Number)]  
  [plus (lhs : AE) (rhs : AE)]  
  [sub (lhs : AE) (rhs : AE)])
```

- A plus and a sub takes two arguments
- Both arguments must be of type AE

Representing the Program

How would you represent these expressions using our abstract syntax?

1

2.3

1 + 2

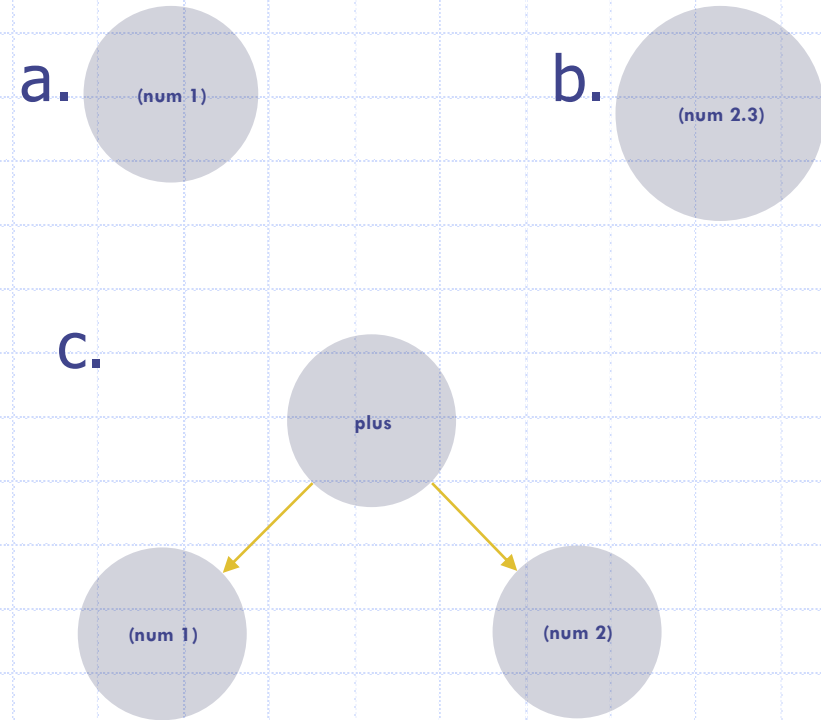
1 + 2 + 3

1 + 2 + 3 + 4

Representing the Program

How would you represent these expressions using our abstract syntax?

- a. 1
- b. 2.3
- c. 1 + 2
- d. 1 + 2 + 3
- e. 1 + 2 + 3 + 4



Representing the Program

How would you represent these expressions using our abstract syntax?

Surface Syntax

1

2.3

1 + 2

1 + 2 + 3

1 + 2 + 3 + 4

Abstract Syntax

(num 1)

(num 2.3)

(plus (num 1) (num 2))

Evaluating the Program

(interpret : (AE -> Number))

(define (interpret an-ae)

(type-case AE an-ae ...

Evaluating the Program

```
(interpret : (AE -> Number))  
(define (interpret an-ae)  
  (type-case AE an-ae  
    [(num n) n]  
    [(plus lhs rhs) (+ (interpret lhs) (interpret rhs))]  
    [(sub lhs rhs) (+ (interpret lhs) (interpret rhs))]  
  ))
```

Testing



Write tests for your evaluator

Testing

Write tests for your evaluator

```
(test (interpret (num 1)) 1)
```

```
(test (interpret (num 2.3)) 2.3)
```

```
(test (interpret (plus (num 1) (num 2))) 3)
```

Testing

Suppressing the reporting of correct tests

◆ Reminder: Use the function:

`(print-only-errors #t)`

Testing

- ◆ Test early, often, and extensively
- ◆ Language implementations are some of the most tested software
- ◆ People will tolerate bugs in other software, but are not as forgiving of bugs in language implementations